Estructura del RTL del RISC-V RV32I segmentado

Antonio Rodríguez

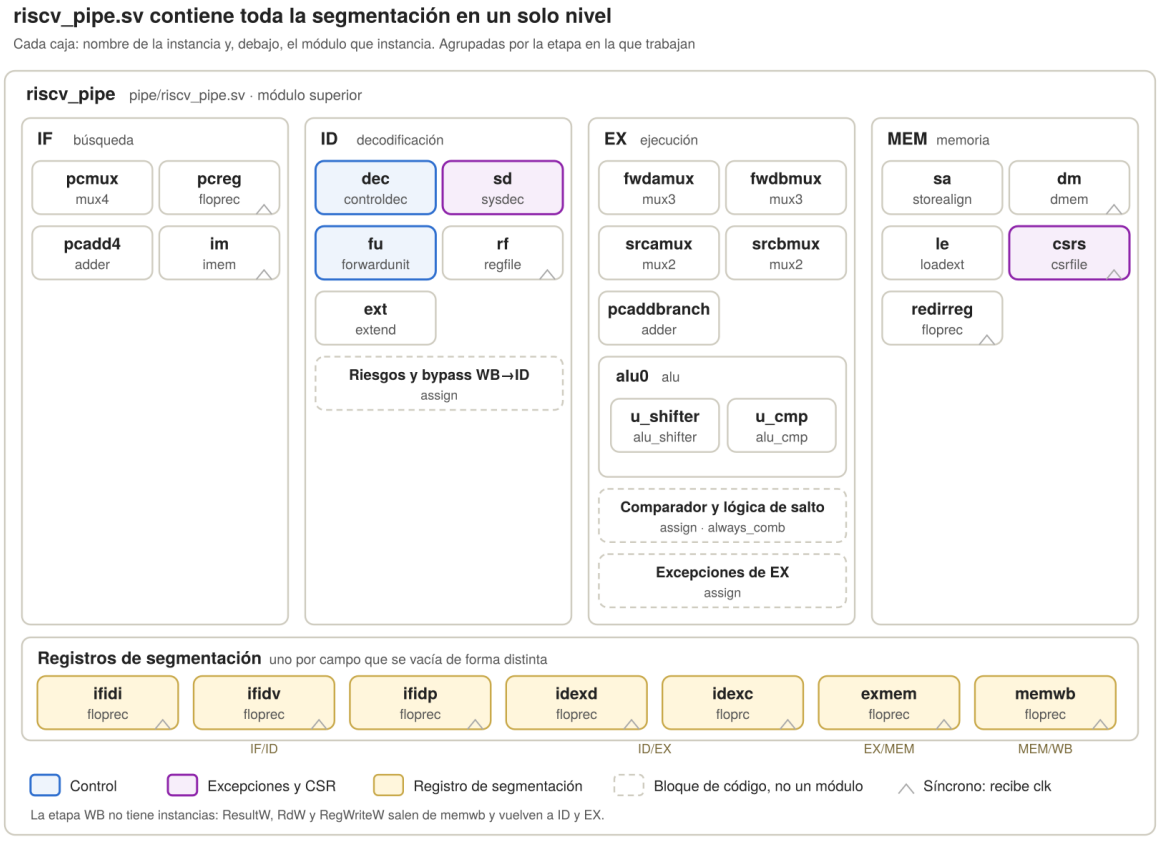
1. Introducción

En este documento se intenta explicar la implementación específica del microprocesador RISC-V segmentado en cinco etapas que hemos hecho nosotros y que puede diferir ligeramente de otras implementaciones que te puedas encontrar. El funcionamiento del control, los riesgos y las excepciones se explica en el documento «Unidad de control del RISC-V RV32I segmentado»; aquí se describe cómo está organizado el código.

El procesador ejecuta RV32I y, además, las instrucciones de acceso a CSR (extensión Zicsr) y las excepciones en modo máquina. Está descrito en 20 módulos de SystemVerilog repartidos en 18 archivos. El módulo superior es `riscv_pipe.sv`, en la carpeta `pipe/`, junto con los cinco módulos nuevos que necesita la segmentación: `floprec`, `floprc`, `forwardunit`, `sysdec` y `csrfile`. El resto (sumador, multiplexores, banco de registros, extensión del inmediato, ALU, memorias, `storealign` y `loadext`) son los mismos archivos de `rtl/` que usa el monociclo.

La diferencia más visible con el monociclo es que ya no hay separación entre `controller` y `datapath`: `riscv_pipe` contiene directamente todas las instancias y toda la lógica que las une, ordenadas por etapas. Así cada señal se puede seguir en el código en el mismo orden en que avanza la instrucción.

2. Jerarquía de módulos



Jerarquía de módulos · 29 instancias bajo riscv_pipe

El árbol solo tiene tres niveles: `riscv_pipe` contiene 27 instancias directas, y solo `alu0` tiene dentro otros dos módulos. El módulo más repetido es `floprec`, con ocho instancias: el PC, `redirreg` y seis de los siete registros de segmentación. `adder`, `mux2` y `mux3` tienen dos instancias cada uno.

Hay además cuatro bloques de lógica que no son módulos, sino asignaciones (`assign` y `always_comb`) escritas dentro de `riscv_pipe`: la unidad de riesgos con el bypass de WB a ID, el comparador con la lógica de salto, y la detección de excepciones en EX. En el esquema aparecen como bloques, pero en el código no tienen instancia.

3. Resumen de módulos

Cada módulo con su archivo, el nombre de sus instancias (todas están en `riscv_pipe`, salvo las dos de `alu`), la etapa en la que trabaja y si es combinacional o tiene parte síncrona.

Módulo	Archivo	Instancias	Etapa	Tipo	Función
`riscv_pipe`	pipe/riscv_pipe.sv	— (módulo superior)	todas	Estructural y lógica propia	Une las cinco etapas; contiene riesgos, forwarding y excepciones
`floprec`	pipe/floprec.sv	pcreg, ifidi, ifidv, ifidp, idexd, exmem, memwb, redirreg	—	Síncrono	Registro con ce, habilitación (en) y borrado (clr)
`floprc`	pipe/floprc.sv	idexc	ID/EX	Síncrono	Registro con ce y borrado, sin habilitación
`mux4`	rtl/mux4.sv	pcmux	IF	Combinacional	Elige el siguiente PC
`adder`	rtl/adder.sv	pcadd4, pcaddbranch	IF, EX	Combinacional	PC + 4 y PC + ImmExt
`imem`	rtl/imem.sv	im	IF	Lectura combinacional; escritura síncrona opcional	Memoria de instrucciones
`controldec`	rtl/controldec.sv	dec	ID	Combinacional	Señales de control de RV32I
`sysdec`	pipe/sysdec.sv	sd	ID	Combinacional	Instrucciones ilegales, ecall, ebreak, mret y CSR
`forwardunit`	pipe/forwardunit.sv	fu	ID	Combinacional	Selección de los multiplexores de forwarding
`regfile`	rtl/regfile.sv	rf	ID / WB	Escritura síncrona, lectura combinacional	Banco de 32 registros (sin reset)
`extend`	rtl/extend.sv	ext	ID	Combinacional	Genera el inmediato de 32 bits
`mux3`	rtl/mux3.sv	fwdamux, fwdbmux	EX	Combinacional	Forwarding de los dos operandos
`mux2`	rtl/mux2.sv	srcamux, srcbmux	EX	Combinacional	Eligen los operandos de la ALU
`alu`	rtl/alu.sv	alu0	EX	Combinacional	Operaciones aritméticas y lógicas
`alu_shifter`	rtl/alu.sv	u_shifter (alu)	EX	Combinacional	Desplazamientos sll, srl, sra
`alu_cmp`	rtl/alu.sv	u_cmp (alu)	EX	Combinacional	Comparaciones slt, sltu
`storealign`	rtl/storealign.sv	sa	MEM	Combinacional	Prepara el dato y la máscara de sb, sh, sw

Módulo	Archivo	Instancias	Etapa	Tipo	Función
`dmem`	rtl/dmem.sv	dm	MEM	Escritura síncrona, lectura combinacional	Memoria de datos
`loadext`	rtl/loadext.sv	le	MEM	Combinacional	Extrae y extiende el dato de lb, lh, lw, lbu, lhu
`csrfile`	pipe/csrfile.sv	csrs	MEM	Escritura síncrona, lectura combinacional	CSR de modo máquina; entrada y salida de excepciones

Los archivos `riscv.sv`, `controller.sv`, `datapath.sv` y `flopr.sv` de `rtl/` son solo del monociclo; el segmentado no los usa.

4. riscv_pipe.sv: el módulo superior

Tiene un parámetro, `INIT_FILE` (por defecto "mem/imem.hex"), que pasa a `imem` para indicar qué programa cargar.

Puertos.

Puerto	Dirección	Para qué sirve
`clk`, `rst_n`	Entrada	Reloj y reset asíncrono activo a nivel bajo.
`ce`	Entrada	Habilitación de reloj. Los registros (PC, segmentación, CSR) y las escrituras en el banco y en `dmem` solo avanzan en los ciclos con `ce = 1`. Con `ce` fijo a 1 es un procesador normal; en la FPGA (`fpga/top_pipe.sv`) un contador genera un pulso de `ce` cada `DIV` ciclos para poder seguir la ejecución en los LED.
`dbg[31:0]`	Salida	`ResultW`: el valor que se escribe en el banco de registros en este ciclo.
`dbg_we`	Salida	`RegWriteW && RdW != 0`: indica que `dbg` es una escritura real, no una burbuja ni una escritura en x0.
`imem_we`, `imem_waddr`, `imem_wdata`	Entrada	Puerto opcional para cargar el programa desde fuera. Sin conectar valen 0 (tienen valor por defecto en la declaración). Además evita que la síntesis trate la memoria de instrucciones como una constante.

A diferencia del monociclo, no saca `PC`, `Instr`, `DataAdr` ni `WriteData`: los testbench observan las señales internas por jerarquía (por ejemplo `dut.PCF`, `dut.InstrD` o `dut.rf`).

Organización. El código está dividido en nueve bloques, marcados con comentarios, en el mismo orden en que avanza la instrucción:

1. **Etapa IF**: `pcmux`, `pcreg`, `pcadd4` e `im`, y la selección `PCSelF` del siguiente PC.
2. **Registro IF/ID**: la unidad de riesgos (`lwStall`, `StallF`, `StallD`, `FlushD`, `FlushE`) y los registros `ifidi`, `ifidv` e `ifidp`.
3. **Etapa ID**: separación de los campos de la instrucción, `dec`, `rf`, el bypass de WB a ID, `ext`, `sd` y los ajustes para las instrucciones CSR.
4. **Registro ID/EX**: `fu` (el forwarding se calcula aquí), `IssueD` y los registros `idexd` e `idexc`.
5. **Etapa EX**: `fwdamux`, `fwdbmux`, `srcamux`, `srcbmux`, `alu0`, `pcaddbranch`, el comparador de saltos, `PCSrcE`, las excepciones de EX y la selección de `WriteDataE`.

6. **Registro EX/MEM**: la selección de `FwdValE`, el anulado `OkE` y `exmem`.
7. **Etapa MEM**: `sa`, `dm`, `le`, `csrs`, la redirección del PC (`RedirM`, `redirreg`) y la selección de `ResultM`.
8. **Registro MEM/WB**: `memwb`.
9. **Etapa WB**: solo las salidas `dbg` y `dbg_we`.

Nombres de las señales. Cada señal lleva al final la letra de la etapa en la que está: F, D, E, M o W. La misma señal se llama `RegWriteD` en ID, `RegWriteE` en EX, `RegWriteM` en MEM y `RegWriteW` en WB, y cada cambio de letra es un registro de segmentación. Las señales que terminan en `x` (`RegWriteDx`, `ResultSrcDx`, `ImmExtDx`, `ForwardADx`, `ExcXE`, `CauseXE`) son versiones modificadas de la señal original, para las instrucciones CSR o las excepciones.

Registros empaquetados. Cada registro de segmentación es un único `floprec` de muchos bits. La entrada se monta con una concatenación y la salida se separa con otra, en el mismo orden:

```
assign idexd_d = {RD1Dop, RD2Dfix, ImmExtDx, PCD, PCPlus4D, rdD,
                  ForwardADx, ForwardBD, ALUSrcD, ASrcD, ALUControlD, funct3D,
                  CauseD, CsrWriteD};
...
assign {RD1E, RD2E, ImmExtE, PCE, PCPlus4E, RdE,
        ForwardAE, ForwardBE, ALUSrcE, ASrcE, ALUControlE, funct3E,
        CauseE, CsrWriteE} = idexd_q;
```

Así hay una sola instancia por registro y se ve de un vistazo qué pasa de una etapa a la siguiente. A cambio, añadir un campo obliga a tocar tres sitios: el ancho del registro y las dos concatenaciones.

5. Registros de segmentación

5.1. floprec y floprc

Son registros de tipo D con ancho parametrizable (`WIDTH`, 32 por defecto), reset asíncrono activo a nivel bajo y entrada `ce`. Solo cambian en un flanco de subida de `clk` con `ce = 1`:

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) q <= '0;
  else if (ce) begin
    if (clr)     q <= '0;      // borrado: tiene prioridad
    else if (en) q <= d;       // habilitación
  end
```

`floprc` es igual pero sin `en`: con `ce = 1` siempre carga `d` salvo que `clr` lo ponga a 0. Las dos versiones existen para que cada registro tenga solo las entradas que usa.

5.2. Contenido de cada registro

Registro	Instancia	Bits	en	clr	Contenido (sin sufijo de etapa)
PC	pcreg (floprec)	32	¬StallF	—	PC
IF/ID	ifidi (floprec)	32	¬StallD	—	Instr
IF/ID	ifidv (floprec)	1	¬StallD	FlushD	Valid (la entrada vale siempre 1)
IF/ID	ifidp (floprec)	64	¬StallD	—	PC, PCPlus4

Registro	Instancia	Bits	en	clr	Contenido (sin sufijo de etapa)
ID/EX	idexd (floprec)	183	1	—	RD1, RD2, ImmExt, PC, PCPlus4, Rd, ForwardA, ForwardB, ALUSrc, ASrc, ALUControl, funct3, Cause, CsrWrite
ID/EX	idexc (floprc)	10	—	FlushE	RegWrite, MemWrite, Branch, Jump, Jalr, ResultSrc, Exc, Mret, Csr
EX/MEM	exmem (floprec)	149	1	—	ALUResult, WriteData, PCPlus4, FwdVal, Rd, RegWrite, MemWrite, ResultSrc, funct3, Exc, Cause, Mret, Csr, CsrWrite, Jalr
MEM/WB	memwb (floprec)	70	1	—	Result, PCPlus4, Rd, RegWrite
—	redirreg (floprec)	1	1	—	RedirM retrasado un ciclo (RedirMd)

Solo IF e ID se pueden detener; EX/MEM y MEM/WB avanzan en todos los ciclos con `ce = 1`. Por eso solo `pcreg` y los tres registros de IF/ID tienen en.

Por qué ID/EX está partido en dos. Para convertir una instrucción en una burbuja basta con poner a 0 las señales que cambian algo: escribir en un registro o en memoria, saltar, provocar una excepción o una detención. Esas señales están en `idexc`, que es el único que tiene `clr`. Los datos y el resto del control van en `idexd`, que nunca se borra: una burbuja lleva datos sin sentido, pero con el control a 0 no tiene ningún efecto. Así el borrado afecta a 10 biestables en lugar de a 193.

Por qué IF/ID tiene un bit de validez. El vaciado de IF/ID no borra la instrucción: solo pone a 0 `ValidD`, y en ID una instrucción no válida no deja pasar su control a EX (señal `IssueD`). El comentario del código lo explica: así `PCSrcE`, que es de las últimas señales en estar lista en cada ciclo, llega a un biestable y no a la entrada de borrado de 32.

Campos que solo sirven para depurar. `PCPlus4` llega hasta MEM/WB solo para que las trazas de simulación puedan mostrar el PC de cada escritura; la síntesis lo elimina porque no se usa.

6. Las etapas en el código

6.1. IF — búsqueda

- `pcmux` (mux4) elige `PCNextF` entre `PCPlus4F` (00), `PCTargetE` (01), `JalrTargetE` (10) y `RedirTargetM` (11). Su selección es `PCSelF = RedirM ? 2'b11 : PCSrcE`.
- `pcreg` guarda el PC si `StallF = 0`.
- `pcadd4` calcula `PCPlus4F` e `im` entrega `InstrF`.

Las direcciones de salto llegan desde etapas posteriores: los saltos se resuelven en EX y las excepciones y `mret`, en MEM.

6.2. ID — decodificación

- Separa los campos `rdD`, `funct3D`, `rs1D` y `rs2D` de `InstrD` con asignaciones continuas.
- `dec` (controldec) genera el control de RV32I, igual que en el monociclo.
- `rf` lee `rs1D` y `rs2D`. Escribe en WB: dirección `RdW`, dato `ResultW` y habilitación `RegWriteW & ce`.
- El **bypass de WB a ID** (`RD1Dfix`, `RD2Dfix`) sustituye la lectura del banco por `ResultW` cuando WB está escribiendo el mismo registro en ese ciclo.

- `ext` genera `ImmExtD`.
- `sd` (sysdec) detecta instrucciones ilegales, `ecall`, `ebreak`, `mret` y CSR, y de sus salidas se forman `ExcD`, `CauseD` y `CsrImmD`.
- Ajustes para las instrucciones CSR: `RD1Dop`, `ImmExtDx`, `RegWriteDx` y `ResultSrcDx`.
- `fu` (forwardunit) calcula `ForwardAD` y `ForwardBD` un ciclo antes de usarlos, y `IssueD` decide si el control entra en EX.

La lógica de detección de riesgos está escrita en el bloque «Registro IF/ID», justo antes de los registros que controla.

6.3. EX — ejecución

- `fwdamux` y `fwdbmux` (mux3) eligen `FwdAE` y `FwdBE` entre el valor leído en ID (00), `ResultW` (01) y `FwdValM` (10).
- `srcamux` elige `SrcAE` entre `FwdAE` y `PCE` (`ASrcE`); `srcbmux` elige `SrcBE` entre `FwdBE` e `ImmExtE` (`ALUSrcE`).
- `alu0` calcula `ALUResultE`. Su salida `zero` queda sin conectar: los saltos usan su propio comparador.
- `pcaddbranch` calcula `PCTargetE = PCE + ImmExtE`; `JalrTargetE = {ALUResultE[31:1], 1'b0}`.
- El comparador (`EqE`, `LtE`, `LtuE`) y un `always_comb` sobre `funct3E` dan `TakenE`, y de ahí `PCSrcE`.
- Detección de excepciones: `AddrLowE`, `MisFetchE`, `MisLoadE`, `MisStoreE`, `ExcXE` y `CauseXE`.
- `WriteDataE` lleva a MEM el dato del store, el operando de una instrucción CSR o el destino de un salto (para `mtval`).
- `FwdValE` elige ya en EX el valor que se podrá reenviar desde MEM, y `OkE = ¬RedirM` anula lo que entra en MEM durante una redirección.

6.4. MEM — memoria

- `sa` (storealign), `dm` (dmem) y `le` (loadext), como en el monociclo, pero con `funct3M` y `ALUResultM[1:0]` en lugar de campos de `Instr`. La escritura en memoria se habilita con `MemWriteM & !ExcM & ce`.
- `TvalM` elige el valor de `mtval` según la causa de la excepción.
- `csrs` (csrfile) lee y escribe los CSR y atiende las excepciones y `mret`.
- `RedirM = ExcM || MretM` y `RedirTargetM = ExcM ? MtvecM : MepcM` redirigen el PC; `redirreg` guarda `RedirM` un ciclo más (`RedirMd`).
- `ResultM` elige el valor que se escribirá en `rd`: `CsrRdataM`, `ReadDataM` o `FwdValM`.

6.5. WB — escritura

No tiene lógica propia. `ResultW`, `RdW` y `RegWriteW` salen de `memwb` y van al banco de registros, al bypass de ID y a los multiplexores de forwarding de EX. `dbg` y `dbg_we` son copias para observar el procesador desde fuera.

7. Módulos nuevos de la segmentación

7.1. forwardunit

Puertos. Entradas `Rs1E`, `Rs2E`, `RdM`, `RdW` (5 bits), `RegWriteM` y `RegWriteW`. Salidas `ForwardAE` y `ForwardBE` (2 bits).

Contenido. Un `always_comb` con la regla clásica, para cada operando por separado: 10 si la instrucción de MEM escribe ese registro (y no es x0), 01 si lo escribe la de WB, y 00 en otro caso. MEM tiene prioridad porque es el valor más reciente.

Los nombres de los puertos son los de la versión clásica, que calcula el forwarding en EX. `riscv_pipe` lo instancia un ciclo antes, en ID, conectando cada puerto con la señal de la etapa anterior:

```
forwardunit fu (.Rs1E(rs1D), .Rs2E(rs2D), .RdM(RdE), .RdW(RdM),
                .RegWriteM(RegWriteE), .RegWriteW(RegWriteM),
                .ForwardAE(ForwardAD), .ForwardBE(ForwardBD));
```

El resultado viaja en `idexd`, así que en EX la selección de `fwdamux` y `fwdbmux` sale directamente de un biestable. El razonamiento está en la sección 7 del documento de control.

7.2. sysdec

Puertos. Entrada `instr[31:0]`. Salidas `Illegal`, `Ecall`, `Ebreak`, `Mret`, `Csr` (instrucción CSR válida) y `CsrWrite` (además escribe el CSR).

Contenido. Decodifica lo que `controldec` no mira:

- `Ecall`, `Ebreak` y `Mret` comparan la instrucción completa con su codificación (0x00000073, 0x00100073 y 0x30200073).
- Un `case` sobre la dirección del CSR dice si existe; las direcciones 0xF1x son de solo lectura.
- `CsrWrite` vale 1 en `csrrw` y `csrrwi`, y en el resto si el campo rs1 (o uimm) no es 0.
- `Csr` vale 1 si el opcode es 1110011, funct3 no es 000 ni 100, el CSR existe y no se intenta escribir uno de solo lectura.
- `Illegal` comprueba, opcode a opcode, que la codificación existe en RV32I con Zicsr (detalle en el documento de control). `wfi` (0x10500073) es legal y se ejecuta como un nop.

7.3. csrfile

Puertos. Para las instrucciones CSR: `csr_en`, `csr_we`, `csr_addr[11:0]`, `csr_op[1:0]` (funct3[1:0]), `csr_wdata` y `csr_rdata`. Para las excepciones: `trap`, `cause[3:0]`, `tval`, `epc`, `mret` y las salidas `mtvec` y `mepc`. Además `clk`, `rst_n` y `ce`.

Contenido.

- Registros internos: `mie` y `mpie` (los dos bits de `mstatus` que se guardan), `mtvec_q` y `mepc_q` (30 bits: los dos bits bajos valen siempre 0), `mcause_int` y `mcause_code`, `mtval` y `mscratch`.
- Lectura combinacional con un `case` sobre `csr_addr`. `misa` es la constante 0x40000100 (RV32I) y los CSR que no se guardan (`mie`, `mip`, `mstatush` e identificadores) devuelven 0.
- `wval` es el valor que se escribiría: `csr_wdata` (01, csrrw), `csr_rdata | csr_wdata` (10, csrrs) o `csr_rdata & ~csr_wdata` (11, csrrc).

- Un `always_ff` con prioridad: primero `trap` (guarda `mepc`, `mcause`, `mtval` y pasa `mie` a `mpie`), luego `mret` (repone `mie`) y por último la escritura de una instrucción CSR.

8. Módulos reutilizados de rtl/

Su funcionamiento se explica en «Estructura del RTL» del monociclo. Lo que cambia es cómo se conectan:

Módulo	Uso en el segmentado
`regfile`	Se instancia con `RESET = 0`: sin reset, para que la síntesis lo implemente en LUTRAM (mucha menos área). Los registros empiezan a 0 por un `initial`. La escritura se habilita con `RegWriteW & ce`. Como la lectura no ve el dato que se escribe en el mismo ciclo, hace falta el bypass de WB a ID.
`controldec`	Sin cambios. `fence` y las instrucciones de sistema (opcode 1110011) tienen una rama vacía: para `controldec` son un nop, y es `sysdec` quien las decodifica.
`alu`	La salida `zero` no se conecta. Los saltos comparan con su propio comparador.
`imem`	Se conecta su puerto de escritura (`clk`, `we`, `waddr`, `wdata`) a los puertos `imem_*` de `riscv_pipe`.
`dmem`	La escritura se habilita con `MemWriteM & !ExcM & ce`: un acceso que provoca una excepción no escribe.
`mux3`, `mux4`	`mux3` se usa en el forwarding (el código 11 nunca se genera); `mux4` elige el PC y ahora usa sus cuatro entradas.
`storealign`, `loadext`	Reciben `funct3M` y `ALUResultM[1:0]` del registro EX/MEM en lugar de campos de `Instr`.
`adder`, `mux2`, `extend`	Sin cambios.

9. Reloj, reset, ce y elementos síncronos

Reciben `clk` el PC, los siete registros de segmentación, `redirreg`, `regfile`, `dmem`, `csrfile` e `imem` (solo su puerto de escritura). En cada flanco de subida con `ce = 1` todos avanzan a la vez y cada etapa pasa a trabajar con la instrucción siguiente. En un ciclo hay hasta cinco instrucciones en marcha, una por etapa.

El periodo de reloj ya no lo marca el recorrido completo de una carga, como en el monociclo, sino la etapa más lenta. Por eso el RTL incluye varias decisiones, explicadas en sus comentarios, para acortar los caminos más largos:

- La selección del forwarding se calcula en ID y llega a EX desde un biestable.
- El valor que se reenvía desde MEM (`FwdValM`) se elige ya en EX y sale de un registro: `ResultSrcM` no está en el camino del forwarding.
- `ResultM` se elige en MEM, así que `ResultW` sale directamente de un registro: no hay multiplexor delante del forwarding desde WB ni del banco de registros.
- Los saltos tienen su propio comparador en lugar de esperar al resultado de la ALU y a la detección de cero.
- Los dos bits bajos de rs1 + inmediato, que deciden si un acceso o un destino de `jalr` está desalineado, salen de un sumador propio de 2 bits (`AddrLowE`).

- El vaciado de IF/ID solo borra `ValidD`, y la anulación por excepción se hace con `IssueD` en lugar de con `FlushE`, para no alargar el camino de `PCSrcE`.
- `RedirM` y `RedirTargetM` salen de registros (`ExcM`, `MretM`, `mtvec` y `mepc`): la redirección solo añade una entrada al multiplexor del PC.

El reset `rst_n` es asíncrono y activo a nivel bajo. Pone a 0 el PC, los registros de segmentación (que quedan llenos de burbujas, porque el control y `ValidD` valen 0), `redirreg` y los CSR. El banco de registros no tiene reset, pero empieza a 0; `dmem` no se reinicia e `imem` conserva el programa.

10. Verificación

- `pipe/tb_pipe1.sv` a `tb_pipe6.sv`: programas de prueba de la segmentación (forwarding, detenciones, saltos…), con la comprobación de los registros al final.
- `pipe/tb_pipe_ce.sv`: el mismo tipo de prueba con `ce = 1` solo un ciclo de cada tres.
- `pipe/tb_pipe_exc.sv`: `exc1` prueba las instrucciones CSR, `fence`, `wfi` y `fence.i`; `exc2` provoca doce excepciones y el manejador anota `mcause`, `mtval` y `mepc` de cada una.
- `compara.sh`: ejecuta el mismo programa en el monociclo (`tools/run_prog.sv`) y en el segmentado (`tools/run_pipe.sv`) y compara las trazas de escrituras en registros (R) y en memoria (M). `fuzz.sh` lo repite con programas aleatorios generados por `tools/gen_random.py`.

Para compilar el segmentado hacen falta los archivos de `rtl/` (salvo los cuatro del monociclo, que no estorban) y los seis de `pipe/`:

```
iverilog -g2012 -o sim/pipe.vvp rtl/*.sv pipe/floprc.sv pipe/floprec.sv \
    pipe/forwardunit.sv pipe/sysdec.sv pipe/csrfile.sv pipe/riscv_pipe.sv \
    pipe/tb_pipe5.sv
```